

Stance measurement for one topic

```
suppressPackageStartupMessages({
  library(here); library(splines); library(viridis)
  library(tidyverse)      # LAST so its verbs win
})
# Namespaced only: ellmer (LLM calls), glue.

st <- list(
  topic = 1L, # THE topic this render processes
  use_openrouter = TRUE, # FALSE to use the OpenAI-compatible endpoint
  dry_run = FALSE, # TRUE: build prompts, print arithmetic, stop
  pilot_n = NULL, # e.g. 20L to test on a subset first

  # One row per pipeline step. base_url_env is the NAME of the .Renviro
  # variable holding that step's endpoint, so different steps can point at
  # different gateways and no URL or key appears in this file. Ignored when
  # st$use_openrouter is TRUE, which routes through llm_openrouter.R instead.
  steps = tibble::tribble(
    ~step, ~base_url_env, ~api_key_env, ~model,
    "proposition", "COMPASS_LARGE_URL", "COMPASS_LARGE_KEY", "MODEL-FOR-
PROPOSITIONS",
    "stance_1", "COMPASS_STANCE_URL", "COMPASS_STANCE_KEY", "MODEL-FOR-
STANCE-1",
    "stance_2", "COMPASS_STANCE_URL", "COMPASS_STANCE_KEY", "MODEL-FOR-
STANCE-2"),

  window_n = 300L, # calls allowed per quota window PER model
  window_wait = 4L * 60L * 60L, # seconds to sleep at the window boundary
  min_theta = 0, # scope floor: drop paragraphs whose modal topic share is
  # below this (0 keeps the census; raise it when AvePP is low)
  min_class_n = 10L, # paragraphs a stance class needs before a trend is fit
  era_width = 5L, # year grouping for the era table
  df_spline = 3L, # spline df for the trend curve
  n_gold = 100L, # paragraphs sent for human coding
  rec_sizes = c(50, 100, 200, 400, 800), # recovery-curve subsample sizes
  rec_reps = 5L
)

theme_set(theme_minimal(base_size = 12))
# Tables must not silently lose rows in the rendered PDF.
options(knitr.table.max.print = Inf, knitr.kable.max_rows = Inf,
  max.print = 1e6)
set.seed(2026)
```

```

p_out <- function(f) here::here("outputs", f)
p_tag <- function(stem, ext = ".csv") # topic-stamped filenames
  p_out(str_c(stem, "_topic", sprintf("%02d", st$topic), ext))
dir.create(here::here("outputs"), showWarnings = FALSE, recursive = TRUE)
source(here::here("code", "llm_source.R"))
if (isTRUE(st$use_openrouter))
  source(here::here("code", "llm_openrouter.R"))
if (isTRUE(st$use_openrouter))
  st$steps$model <- open_router_models[st$steps$step]

```

Frame: every paragraph in this topic

02_topics.qmd assigned every paragraph its modal STM topic. This script measures stance on **all** paragraphs of one topic, so there is no sampling and no sampling error: the proportions below are exact for the corpus. The budget is handled by time rather than by design, classifying 300 paragraphs per quota window and sleeping until the reset, which makes the method portable to any corpus size without retuning a sample.

```

paragraph_theta <- readr::read_csv(p_out("paragraph_theta.csv"),
                                   show_col_types = FALSE)
topic_summary <- readr::read_csv(p_out("topic_summary.csv"),
                                  show_col_types = FALSE)
stopifnot("02_topics.qmd must supply modal topic and text" =
  all(c("para_uid", "atom_id", "Year", "text", "topic",
        "theta_max") %in% names(paragraph_theta)))

frame_all <- paragraph_theta |>
  dplyr::filter(topic == st$topic) |>
  dplyr::select(para_uid, atom_id, Year, n_tokens, text, theta_max)
# Modal assignment is fuzzy when AvePP is low: a paragraph can be "in" this
# topic at a share of 0.2 while really being about something else, and such
# paragraphs come back irrelevant. min_theta removes them as a documented
# scope restriction; the default of 0 keeps the full census.
frame <- dplyr::filter(frame_all, theta_max >= st$min_theta)
if (nrow(frame) == 0) stop("No paragraphs for topic ", st$topic,
  " at min_theta = ", st$min_theta, ".")
writeLines(glue::glue("Scope: {nrow(frame)} of {nrow(frame_all)} paragraphs \
  have a topic share of at least {st$min_theta} (median share \
  {round(stats::median(frame_all$theta_max), 2)})."))

```

Scope: 104 of 104 paragraphs have a topic share of at least 0 (median share 0.49).

```

n_windows <- ceiling(nrow(frame) / st>window_n)
glue::glue("Topic {st$topic}: {nrow(frame)} paragraphs from \
  {dplyr::n_distinct(frame$atom_id)} comments across \
  {dplyr::n_distinct(frame$Year)} years. \

```

```
{nrow(frame)} calls per model = {n_windows} quota window(s), \\
about {round(n_windows * st$window_wait / 3600, 1)} hours \\
unattended; the two raters run their windows independently.")
```

Topic 1: 104 paragraphs from 34 comments across 34 years. 104 calls per model = 1 quota window(s), about 4 hours unattended; the two raters run their windows independently.

Codebook: label, description, proposition

The codebook CSV written by `02_topics.qmd` is the source of truth. Labels and descriptions come from there and are never redrafted here. The only field this script owns is the **proposition**, drafted once if the cell is empty and then left alone. All three fields enter the stance prompt, so **edit before classifying**: changing them afterward means the stored labels were produced by a different instrument. The script compares file times and warns if that has happened, and it stops by name if the topic model was refit after these labels were produced, since a refit can renumber topics.

The proposition is the measurement target and must be a **contestable directional claim** (“funding for X should increase”), not a topic restatement (“this is about funding”), because a non-contestable proposition drives every paragraph to neutral and quietly destroys the measurement.

```
cb_path <- p_out("topic_codebook.csv")
theta_in <- p_out("paragraph_theta.csv")
# The codebook must describe the CURRENT fit: a refit in 02_topics.qmd can
# renumber topics, and a stale codebook would attach labels to the wrong ones.
check_fingerprint(cb_path, theta_in, strict = FALSE)
if (!file.exists(cb_path))
  stop("No topic_codebook.csv in outputs/. Render 02_topics.qmd first; it drafts
",
      "the labels and descriptions this script needs.")
codebook <- readr::read_csv(cb_path, show_col_types = FALSE)
if (!st$topic %in% codebook$topic)
  stop("Topic ", st$topic, " is not in the codebook.")

# Labels and descriptions come from 02_topics.qmd and are never redrafted here.
# Only the proposition is this script's business, and only if it is empty.
if (is.na(codebook$proposition[codebook$topic == st$topic]) ||
    !nzchar(str_squish(codebook$proposition[codebook$topic == st$topic] %||%
"")) {
  row <- codebook |> dplyr::filter(topic == st$topic)
  sys <- build_prompt(
    role = str_c("You are a survey methodologist drafting stance items for a ",
      "public research program."),
    task = str_c("From this topic's label and description, draft ONE ",
      "proposition that a commenter could plausibly support or ",
      "oppose. Before answering, test it: if a paragraph could not ",
      "disagree with it, rewrite it."),
    rules = c("A proposition states a direction, not a subject.",
```

```

      "Use only the label and description shown; invent no programs.",
      "Keep it under 20 words and free of loaded language.",
      "Do not restate the topic label as a proposition."),
  output = "One proposition.")
schema <- ellmer::type_object(
  proposition = ellmer::type_string("A contestable directional claim.")
draft <- call_one(make_chat("proposition", sys, st$steps, st$use_openrouter),
  str_c("Label: ", row$label,
        "\nDescription: ", row$description), schema)
if (is.null(draft)) stop("Proposition drafting call failed for topic ",
  st$topic, ".")
codebook$proposition[codebook$topic == st$topic] <- draft$proposition
readr::write_csv(codebook, cb_path)
# Stop here. The draft is unreviewed, and classifying with it would spend a
# full census of calls on a measurement target nobody has read. Edit the
# proposition, then render again; the next render finds it filled and runs.
writeLines(glue::glue("DRAFT proposition written for topic {st$topic}: \\
  \"{draft$proposition}\". Review and edit it in {basename(cb_path)}, \\
  then render again. Nothing was classified."))
knitr::knit_exit()
}

```

DRAFT proposition written for topic 1: “National reconstruction efforts should prioritize correcting past mistakes over preserving existing successes.”. Review and edit it in `topic_codebook.csv`, then render again. Nothing was classified.

```

cb <- codebook |> dplyr::filter(topic == st$topic)
stopifnot("codebook row must be complete" =
  all(nzchar(c(cb$label, cb$description, cb$proposition))))
cb |>
  dplyr::mutate(ttl = str_c("Topic ", topic, ": ", label)) |>
  tbl_records("ttl", c("description", "proposition"),
    caption = "Codebook row driving this render. All three fields
enter the stance prompt; the proposition is the sole measurement target.")

```

Codebook row driving this render. All three fields enter the stance prompt; the proposition is the sole measurement target.

Topic 1: Reconstruction Task - description: The paragraphs discuss the need for national reconstruction, restoration, and readjustment after a period of failure and decay, with a focus on correcting evil without impairing the good. - proposition: National reconstruction efforts should prioritize correcting past mistakes over preserving existing successes.